DOCUMENTACIÓN TÉCNICA OFICIAL

# SNNU AI Web Portal

Portal Científico con Traducción por IA Local (Zero-Data-Leakage), Recorrido Virtual WebXR/Unity y Catálogo Bibliométrico Automatizado

**Institución:** Universität des Saarlandes (Facultad de Medicina, Homburg) & HTW Saar (Facultad de Ingeniería, Saarbrücken)

**Unidad de Investigación:** Systems Neuroscience & Neurotechnology Unit (SNNU)

**Stack Tecnológico:** React 19 + Vite 8 SPA | FastAPI (Python 3.10+) | SQLite | Cloudflare Workers | Motor LLM Ollama Local

**Ruta en Disco:** `/home/hairex/Documentos/Drive/Dokumente/Antigravity/SNNU`

**Fecha:** Octubre 2026

## 1. Resumen Ejecutivo y Misión del Proyecto

**SNNU** (*Systems Neuroscience & Neurotechnology Unit*) es el portal web interactivo oficial de la unidad de investigación interdisciplinaria compartida entre la **Facultad de Medicina de la Universität des Saarlandes (UdS)** en Homburg/Saar y la **Facultad de Ingeniería de la Hochschule für Technik und Wirtschaft des Saarlandes (HTW Saar)** en Saarbrücken, Alemania.

Bajo la dirección del **Prof. Dr. Daniel J. Strauss** y la **Dra. Farah I. Corona-Strauss**, la unidad desarrolla investigación pionera en neurociencia de sistemas a múltiples escalas, neuromodelado computacional, neuroergonomía, neuroimagen y tecnologías de asistencia auditiva y sensorial.

### Pilares de Innovación Tecnológica del Portal

1. **Traducción Dinámica por IA Local con Cero Fuga de Datos (Zero-Data-Leakage):** Cumple con las estrictas regulaciones europeas de privacidad clínica y médica (GDPR). En lugar de enviar textos a servicios comerciales como Google Translate, el sistema integra modelos de código abierto (`translategemma:4b` o `qwen2.5:1.5b`) mediante **Ollama** ejecutado de forma local o en la red privada de la universidad. El backend preserva intactas las etiquetas HTML, enlaces, fechas, nombres propios de investigadores y referencias académicas.
2. **Entornos Virtuales Inmersivos WebXR y Unity WebGL:** Recorrido virtual 360° interactivo de los laboratorios y del sistema de estimulación acústica inmersiva de 46 altavoces (*46-Speaker System*), respaldado por un proxy edge en Cloudflare Workers para streaming de assets binarios.
3. **Pipeline Bibliométrico Automatizado:** Herramientas ETL que auditan publicaciones científicas contra las APIs de **Crossref** y **OpenAlex**, resolviendo DOIs y descargando copias locales Open Access en PDF vinculadas a los perfiles de los autores.

## 2. Arquitectura General del Sistema

El portal SNNU se divide en cuatro capas desacopladas que garantizan resiliencia y alta velocidad:

| Capa | Tecnología Principal | Responsabilidades |
|---|---|---|
| **Frontend SPA** | React 19, Vite 8, CSS3 Moderno | Aplicación de página única (SPA). Selector de 15 idiomas, Dock flotante de navegación, vistas detalladas de 6 laboratorios, modal biográfico de investigadores con filtrado canónico de publicaciones y reproductor Unity |

| Capa | Tecnología Principal | Responsabilidades |
| --- | --- | --- |
| | | WebGL. |
| **Backend REST** | FastAPI, Uvicorn, SQLAlchemy | API asíncrona de alto rendimiento. Orquesta el parseo y segmentación de HTML, traducción por lotes con Ollama, caché LRU en memoria por idioma, y almacenamiento de configuración en SQLite (`snnu.db`). |
| **Motor de IA Local** | Ollama Server (Local / LAN) | Inferencia determinista de traducción con esquemas JSON estructurados y semáforos de concurrencia limitada. |
| **Edge Infrastructure** | Cloudflare Workers (`worker.js`) | Sirve el frontend compilado (`frontend/dist`) y actúa como proxy inverso con streaming CORS y soporte HTTP Range para archivos pesados de Unity (`.data`, `.wasm`) y videos 360 en MP4. |

## 3. Estructura del Repositorio

```
SNNU/

├── backend/                      # API REST y motor de traducción

│   ├── base_data/                # Estructuras de datos base del portal (Python dicts)

│   │   ├── __init__.py           # Agregador central INITIAL_PAGE_DATA

│   │   ├── hero.py               # Datos de cabecera y video principal

│   │   ├── home.py               # Misión, 3 pilares, métodos y timeline

│   │   ├── nav.py                # Etiquetas del menú y dock de navegación

│   │   ├── news.py               # Noticias y logros científicos

│   │   ├── people.py             # Directorio del equipo, fotos y biografías

│   │   ├── research_site.py      # Datos detallados de los 6 laboratorios

│   │   └── sections/             # Publicaciones, proyectos, docencia, preprints, vacantes

│   ├── main.py                   # Servidor FastAPI y pipeline de traducción

│   ├── models.py                 # Modelos ORM (School, Language, Audience)

│   ├── schemas.py                # Esquemas de validación Pydantic
```

```
│   ├── database.py                # Conexión SQLAlchemy a SQLite
│   ├── seed.py                    # Inicializador de datos en base de datos
│   └── snnu.db                    # Base de datos SQLite
├── frontend/                      # Aplicación SPA cliente (React 19 + Vite 8)
│   ├── public/                    # Activos estáticos públicos
│   │   ├── Build/                 # Binarios de Unity WebGL (UnityLoader.js, data, wasm)
│   │   ├── image/                 # Fotografías de personal y laboratorios
│   │   ├── publication/           # Artículos científicos Open Access en PDF
│   │   └── webxr/                 # Entorno interactivo para visores VR
│   ├── src/                       # Código fuente React
│   │   ├── components/            # HomePage, ProfileModal, ResearchLabDetail
│   │   ├── App.jsx                # Componente raíz, orquestador de estado e i18n
│   │   ├── App.css                # Estilos globales y barra dock
│   │   └── main.jsx               # Montaje de la aplicación React
│   ├── package.json               # Dependencias (React 19.2.7, Vite 8.1.1)
│   └── vite.config.js             # Configuración de empaquetado Vite
├── tools/                         # Scripts de automatización y ETL
│   ├── audit_publications.py      # Scraper/API Crossref + OpenAlex -> manifest.csv
│   └── sync_people_photos.py      # Sincronización de retratos desde el sitio universitario
├── manifest.csv                   # Catálogo maestro de publicaciones con estado PDF
├── papers.csv                     # Lista cruda de DOIs a auditar
├── worker.js                      # Cloudflare Worker para enrutamiento y proxy de assets
├── wrangler.jsonc                 # Configuración de despliegue en Cloudflare Workers
└── start_demo.sh                  # Script de arranque simultáneo de servicios
```

# 4. Frontend: Componentes e Internacionalización (React 19)

## 4.1 Estado Global y Orquestación (`App.jsx`)

- **Estado de Datos (`page`):** Árbol jerárquico de la página provisto por el backend (`/default-data` o `/generate`). Si el backend no responde, el frontend se degrada elegantemente manteniendo el contenido en inglés sin romper la UI.
- **Selector de Idioma Dinámico (15 Idiomas):** Soporta Inglés (`en`), Alemán (`de`), Español (`es`), Francés (`fr`), Italiano (`it`), Portugués (`pt`), Chino (`zh`), Japonés (`ja`), Coreano (`ko`), Ruso (`ru`), Árabe (`ar`), Turco (`tr`), Polaco (`pl`), Holandés (`nl`) y Sueco (`sv`).
- **Ghost Title:** Barra de navegación fija superior que aparece dinámicamente cuando el título de la sección activa se desplaza fuera del campo de visión.
- **Normalización de Autores (`canonicalAuthorName`):** Algoritmo de desambiguación ortográfica que agrupa variantes como `"d strauss"`, `"d j strauss"` y `"daniel j strauss"` bajo la identidad única `"Daniel J. Strauss"` para indexar correctamente su producción bibliográfica en el modal de perfil.

## 4.2 Componentes de Vista Clave

- **`HomePage.jsx`:** Muestra los tres pilares de la unidad:
    1. *Multiscale Systems Neuroscience:* Dinámica neural desde el nivel celular hasta redes corticales.
    2. *Quantitative Neuromodeling:* Modelos biofísicos y simulaciones matemáticas de procesamiento auditivo y sensorial.
    3. *Neuroimaging:* Técnicas no invasivas de EEG de alta densidad y resonancia funcional.

  Incluye la línea de tiempo histórica (2005: University Hospital Homburg, 2012: Technikum Saarbrücken, 2013: Haptics Lab).
- **`ResearchLabDetail.jsx`:** Vista individual para los 6 laboratorios especializados:
    - **Translational Lab:** Dirigido por la Dra. Corinna Martin (aplicación clínica de neurotecnologías).
    - **Mindscan Lab:** Dirigido por la Dra. Farah I. Corona-Strauss (monitoreo cerebral no invasivo).
    - **Multiscale Lab:** Dirigido por el Dr. Lars Haab (neuromodelado y procesamiento de señales).
    - **Digital Realities Lab:** Dirigido por Tobias Grün, M.Sc. (realidad virtual e interacción).
    - **Neurocognitive Haptics Lab:** Percepción táctil y neuroergonomía háptica.
    - **Neurocognitive Driving Test Field:** Evaluación neurocognitiva en simuladores de conducción.
- **`ProfileModal.jsx`:** Ficha modal detallada de cada investigador con foto oficial, afiliación institucional, biografía y lista filtrable de publicaciones científicas con descarga de PDF.
- **Visor Unity WebGL (Recorrido 360°):** Montaje de lienzo WebGL interactivo alimentado mediante el proxy de Cloudflare Workers y enlace al entorno WebXR para cascos Oculus/Meta Quest.

# 5. Backend: Motor de Traducción Determinista (FastAPI + Ollama)

## 5.1 Arquitectura del Pipeline de Traducción (`backend/main.py`)

El núcleo más sofisticado del sistema es su capacidad de traducir árboles JSON complejos sin romper la sintaxis HTML interna ni filtrar datos protegidos:

``` Árbol JSON Original │ ▼ collect_translatable_values() ──► Excluye claves fijas (id, urls, emails, fotos, autores) │ y nombres propios de investigadores ▼ split_html_text() ──► Separa el texto prosa de etiquetas HTML () │ Crea un template posicional de reconstrucción ▼ request_translations() ──► Envía lote JSON a Ollama con Structured Outputs Schema: │ {"translations": ["texto1", "texto2", ...]} ▼ [Validación y Bisección] ──► Si Ollama falla o altera el número de elementos: │ Bisección recursiva (divide el lote en 2 y reintenta) ▼ rebuild_html_text() ──► Reinyecta el texto traducido dentro de los tags HTML │ originales sin modificar clases, atributos ni IDs ▼ Cache LRU + Respuesta JSON ```

> **Innovaciones Técnicas del Algoritmo**

- **Aislamiento de Metadatos (`NON_TRANSLATABLE_KEYS`):** Campos como `id`, `icon`, `videoSrc`, `src`, `url`, `href`, `email`, `photo`, `phone`, `year` y `authors` jamás se envían a inferencia.

- **Preservación de Sintaxis HTML:** Si un texto contiene `<a href="/pdf/doc.pdf">Read Paper</a>`, el modelo solo recibe `"Read Paper"`; la etiqueta y su ruta quedan resguardadas en memoria.

- **Resiliencia por Bisección (Divide & Conquer):** Si Ollama genera un JSON malformado o devuelve una cantidad incorrecta de traducciones para un lote de $N$ elementos, la función divide el lote en dos mitades y procesa cada subconjunto de forma independiente hasta aislar la cadena conflictiva.

- **Caché LRU en Memoria:** Mantiene en RAM las versiones completas traducidas de los últimos idiomas solicitados, evitando llamadas repetitivas al modelo de lenguaje.

## 5.2 Endpoints de la API

| Método | Endpoint | Descripción | Código de Éxito |
|---|---|---|---|
| `GET` | `/default-data` | Retorna el árbol de datos maestro original en inglés (`INITIAL_PAGE_DATA`). | 200 OK |
| `POST` | `/generate` | Procesa o recupera de caché la traducción de la página para el idioma especificado. | 200 OK (502 si falla Ollama) |
| `GET` | `/health` | Verifica el estado del servicio FastAPI y el host/modelo activo | 200 OK |
```

| Método | Endpoint | Descripción | Código de Éxito |
|---|---|---|---|
| | | de Ollama. | |

## 6. Infraestructura Edge: Cloudflare Worker (`worker.js`)

El archivo `worker.js` actúa como proxy inteligente y servidor de contenido estático:

- **Alojamiento Frontend:** Sirve los archivos estáticos de la SPA generados en `frontend/dist` configurados en `wrangler.jsonc`.
- **Proxy Streaming de Assets Unity (CORS Bypass):**
  - `/unity/data`: Transmite `VirtualTourVRweb.data` desde GitHub Releases con cabeceras `Access-Control-Allow-Origin: *` y `Content-Type: application/octet-stream`.
  - `/unity/wasm`: Transmite `VirtualTourVRweb.wasm` con cabecera `Content-Type: application/wasm`.
- **Streaming de Video 360 con Soporte HTTP Range:** Las rutas `/unity/video360`, `/unity/crimson` y `/unity/cereal` proxifican videos MP4 de alta resolución manejando solicitudes `OPTIONS` (Preflight) y propagando cabeceras `Range`, `Content-Range` y `Content-Length` para permitir rebobinado y streaming sin pausas en visores WebXR.

## 7. Herramientas de Auditoría y ETL (`tools/`)

- **`audit_publications.py`:** Script de auditoría bibliométrica que toma `papers.csv`, consulta las APIs de **Crossref** y **OpenAlex**, descarga copias Open Access en PDF a `frontend/public/publication/` con nombres basados en hash SHA-1, y genera automáticamente el módulo Python `backend/base_data/sections/publication.py`.
- **`sync_people_photos.py`:** Script de sincronización que descarga retratos oficiales de los investigadores desde el portal institucional de la Universidad del Sarre y genera `people_photos.csv`.
- **`restore_publications.py`:** Utilidad de contingencia para regenerar las estructuras Python a partir de `manifest.csv` sin requerir conexión a internet.

## 8. Guía de Ejecución y Despliegue

### 8.1 Ejecución Local Automatizada

```
# Levantar backend y frontend simultáneamente con el script incluido

./start_demo.sh
```

## 8.2 Ejecución Manual por Terminales

```
# Terminal 1 — Backend FastAPI

cd backend

source venv/bin/activate

export OLLAMA_HOST="http://localhost:11434"

export OLLAMA_MODEL="translategemma:4b"

uvicorn main:app --host 0.0.0.0 --port 8000 --reload



# Terminal 2 — Frontend React

cd frontend

npm install

npm run dev
```

## 8.3 Despliegue en Cloudflare Workers

```
cd frontend

npm run build

cd ..

npx wrangler deploy
```